

4 D. Abstract_Angajat

Sa se defineasca o ierarhie de clase care urmareste sa efectueze calculul salarial pentru doua categorii de angajati: *Inginer* si *Muncitor*. Pentru a trata unitar angajatii se va construi o clasa abstracta **Angajat** (2,5p) care va contine, ca atribut: marca si numele angajatului iar ca metode:

- Constructori (0,5);
- Functii de acces (0,5);
- Metoda abstracta de calcul salariu (0,5);
- Operatorul << pentru afisarea unui anagajat in forma: marca, nume si salariul (1).

Clasa **Inginer** (2p) in relatie de tip *is a* cu angajatul are ca atribut: numar ore, salariul orar si sporul in timp ce, clasa **Muncitor** (2p), in acelasi tip de relatie cu angajatul va avea ca atribut: numar piese executate si manopera per piesa. Se vor defini constructori (0,5), functii de acces corespunzatoare (0,5) si se va concretiza calculul salarial (1), in ambele clase.

Sa se genereze obiecte ale claselor definite iar acestea sa se stocheze si sa se gestioneze in mod unitar printr-un container secvential din STL (1p). Se va defini o functie independenta care va primi ca parametru colectia anterior definita si va afisa statul de plata prin afisarea angajatilor si se va identifica tipul angajatului care se va afisa sub forma **I** pentru inginer, respectiv **M** pentru muncitor, folosind mecanismul RTTI (1,5p).

Exemplu:

afisare_stat(ob_colectie); **va afisa:**

I: 100, Vasilica, 3422

I: 200, Adam, 2244

M: 300, Elena, 1600

M: 550, Virgil, 1550